Rapport technique : mise en place d'une infrastructure PKI

1. Introduction

Pour ce projet, on a construit une petite PKI avec OpenSSL (option A) : une autorité de certification racine hors ligne, une autorité intermédiaire, des certificats pour un serveur web et pour un client, puis une connexion sécurisée entre les deux avec du mTLS. On a aussi testé la révocation d'un certificat.

Le projet s'est déroulé en groupe, sur deux jours et demi. L'idée n'était pas seulement de faire marcher des commandes, mais de comprendre pourquoi une PKI est organisée comme ça : pourquoi plusieurs niveaux d'autorités, pourquoi protéger les clés privées, pourquoi limiter la durée de vie des certificats. Dans ce rapport, on explique d'abord comment on a monté la PKI, puis comment on l'a utilisée pour sécuriser un site, puis comment on a testé la révocation. On termine par nos choix de sécurité et les difficultés rencontrées.

2. Mise en place de la PKI

On a travaillé sur une machine Linux Debian avec OpenSSL. On a d'abord créé 12 dossiers pour ranger la CA racine, la CA intermédiaire, le serveur et le client. Les dossiers qui contiennent des clés privées sont protégés (permissions 700, donc accessibles seulement à l'administrateur).

Cette organisation en dossiers sert à ne pas mélanger les rôles. La CA racine a son propre dossier avec seulement sa clé et son certificat, alors que la CA intermédiaire a besoin de plus de choses (demandes de signature, certificats émis, listes de révocation). Les dossiers serveur et client contiennent les fichiers qui seront utilisés plus tard pour la partie web. On a vérifié à la fin de cette étape que tout existait bien et que les permissions étaient correctes.

Le tableau suivant résume le contenu de chaque dossier.

Dossier	Contenu	Permissions
root-ca	clé privée et certificat de la CA racine	700 sur le dossier de la clé
intermediate-ca	clé, certificat, demandes de signature, listes de révocation, certificats émis	700 sur le dossier de la clé
server	clé, demande et certificat du serveur web	à définir selon l'usage
client	clé, demande et certificat du client	à définir selon l'usage

Ensuite on a créé la CA racine : une clé privée RSA de 4096 bits protégée par un mot de passe (AES-256) et en lecture seule (permissions 400), puis un certificat auto-signé avec les extensions d'une autorité de certification. On l'a vérifié avec OpenSSL.

La CA racine est la base de toute la confiance : tous les autres certificats dépendent d'elle. C'est pour ça qu'on a choisi une clé plus grande (4096 bits) et une durée de vie longue de 20 ans. Comme elle est auto-signée, personne d'autre ne garantit son identité : il faut donc que ceux qui l'utilisent décident de

lui faire confiance, par exemple en l'ajoutant à leur machine ou à leur navigateur. Elle ne sert qu'à signer l'intermédiaire, puis on peut la mettre de côté.

Pour la CA intermédiaire, on a généré sa clé et une demande de signature (CSR), que la CA racine a signée. Le certificat a les extensions suivantes : CA:TRUE (c'est bien une autorité), pathlen:0 (elle ne peut pas créer d'autres autorités), keyCertSign et cRLSign (elle peut signer des certificats et des listes de révocation). La vérification confirme que l'intermédiaire est bien signée par la racine : la chaîne de confiance est bonne.

Le paramètre pathlen:0 est un détail important : il empêche l'intermédiaire de créer une autre autorité en dessous d'elle. Ça limite les dégâts si elle est piratée, car elle ne pourra signer que des certificats finaux. On a aussi regroupé les certificats de l'intermédiaire et de la racine dans un seul fichier de chaîne, que le serveur web utilisera pour que les clients puissent vérifier toute la chaîne.

L'intérêt d'avoir une intermédiaire, c'est de pouvoir garder la clé de la racine hors ligne. Si l'intermédiaire est compromise, on la révoque et on en refait une, sans tout recommencer.

3. Certificats serveur et client

On a créé un certificat pour le serveur web et un pour le client, tous les deux signés par la CA intermédiaire (jamais directement par la racine). Le certificat serveur contient le nom du site dans le champ SAN, sinon les navigateurs le refusent. Le certificat client a l'usage « authentification client ». On les a vérifiés par rapport à la chaîne de confiance. [capture à insérer]

Pour chaque certificat, le principe est le même : on génère d'abord une clé privée, puis une demande de signature (CSR) qui contient le nom et la clé publique, et on envoie cette demande à la CA intermédiaire pour qu'elle la signe. La clé privée ne quitte jamais la machine de son propriétaire, seule la demande circule.

Le certificat du serveur a une clé RSA de 2048 bits et dure environ un an. Il est limité à l'usage « serveur web » et contient le nom du site dans le SAN. Celui du client est du même type, mais limité à l'usage « client », et il contient le nom de l'utilisateur. Ces limitations d'usage sont importantes : un certificat client ne peut pas servir à se faire passer pour un serveur, et inversement.

Pour comparer, voici les différences principales entre les deux certificats.

	Certificat serveur	Certificat client
Signé par	CA intermédiaire	CA intermédiaire
Clé	RSA 2048 bits	RSA 2048 bits
Durée	environ 1 an	environ 1 an
Usage autorisé	serveur web	client
Identité	nom du site (SAN)	nom de l'utilisateur

4. Application web en mTLS

On a utilisé nginx. Le serveur présente son certificat et exige en plus un certificat client signé par notre CA. Sans certificat client, la connexion est refusée. Avec le bon certificat, la page s'affiche. [capture à insérer]

Dans la configuration de nginx, on a indiqué le certificat et la clé du serveur, puis le fichier de chaîne de notre CA pour qu'il puisse vérifier les certificats des clients. On a activé l'option qui rend le certificat client obligatoire, et on a limité les protocoles à TLS 1.2 et 1.3. Pour pouvoir tester avec le nom du site, on a ajouté une ligne dans le fichier hosts de la machine, et on a indiqué à notre navigateur ou à notre outil de test qu'il devait faire confiance à notre CA racine.

On a fait trois tests : sans certificat client (refusé), avec un certificat client signé par notre CA (accepté), et avec un certificat qui ne vient pas de notre CA (refusé aussi). Cela montre que le serveur ne se contente pas de chiffrer la connexion : il vérifie aussi qui se connecte, et c'est ça la différence entre du HTTPS classique et du mTLS.

5. Révocation et test

On a révoqué le certificat du client avec la CA intermédiaire, généré une liste de révocation (CRL) et configuré nginx pour la lire. Après avoir rechargé nginx, le même certificat client est refusé alors qu'il marchait avant. Ça prouve que la révocation fonctionne. On n'a pas fait OCSP : la CRL suffit pour ce projet. [capture à insérer]

Quand la CA intermédiaire signe un certificat, elle l'écrit dans sa base de certificats émis. Révoquer un certificat, c'est changer son état dans cette base. La CRL est ensuite générée à partir de cette base et signée par la CA, pour que personne ne puisse en fabriquer une fausse. Le serveur nginx lit cette liste et refuse tous les certificats qui y figurent.

Le test s'est fait en deux temps. Avant la révocation, le certificat client permettait d'accéder au site. Après la révocation et le rechargement de nginx, le même certificat, avec la même clé, était refusé. C'est la preuve que c'est bien la révocation qui a bloqué l'accès, et pas une autre erreur.

Au final, on a testé trois cas avec le serveur mTLS.

Test	Résultat attendu	Résultat obtenu
Sans certificat client	refusé	à compléter
Certificat client valide	accepté	à compléter
Certificat client révoqué	refusé	à compléter

La CRL a quand même une limite : elle n'est pas mise à jour toute seule, il faut la régénérer et la recopier sur le serveur à chaque révocation. Avec OCSP, le serveur interroge la CA en direct, ce qui est plus pratique mais demande de mettre en place un service en plus. Pour un projet de cette taille, la CRL était plus simple.

6. Nos choix et notre politique de certification

Clés des CA en RSA 4096 bits : elles durent longtemps (20 ans pour la racine, 10 ans pour l'intermédiaire) et signent tout le reste, donc on a pris large. Clés du serveur et du client en RSA 2048 bits : c'est suffisant, plus rapide, et ces certificats sont renouvelés souvent (environ 1 an). Signatures en SHA-256, parce que SHA-1 n'est plus considéré comme sûr. Sur le serveur, seulement TLS 1.2 et 1.3.

Pour la gestion des clés : clés privées en lecture seule, clés des CA protégées par mot de passe, clé de la racine gardée hors ligne. Seul l'administrateur de la PKI peut demander des certificats, et la CA intermédiaire publie la CRL à chaque révocation. On aurait aussi pu utiliser des clés ECDSA, plus légères, mais RSA était plus simple à manipuler avec OpenSSL.

Voici, en résumé, les règles de notre politique de certification. La CA racine ne signe que l'intermédiaire et reste hors ligne. La CA intermédiaire signe les certificats du serveur et des clients et publie la liste de révocation. Les certificats finaux durent environ un an, pour limiter le risque en cas de vol de clé, et sont renouvelés avant leur expiration. En cas de problème (clé volée, utilisateur parti), le certificat est révoqué et la CRL est mise à jour. Enfin, la machine qui héberge la PKI doit être à jour et accessible uniquement aux administrateurs, avec seulement les services nécessaires.

Ce tableau reprend nos choix cryptographiques et leurs durées de vie.

Élément	Algorithme	Durée de vie	Pourquoi
CA racine	RSA 4096 bits	20 ans	elle signe tout le reste et reste hors ligne
CA intermédiaire	RSA 4096 bits	10 ans	elle signe tous les jours, donc plus exposée
Certificat serveur	RSA 2048 bits	environ 1 an	renouvelé souvent, plus rapide
Certificat client	RSA 2048 bits	environ 1 an	même raison
Signatures	SHA-256		SHA-1 n'est plus sûr

7. Difficultés et conclusion

Les fichiers de configuration d'OpenSSL ont été le plus compliqué à comprendre, avec leurs nombreuses sections. Il ne faut pas oublier le SAN dans le certificat serveur, ni de recharger nginx après la mise à jour de la CRL.

Au final, on a une PKI qui marche de bout en bout et on a vu concrètement comment fonctionne une chaîne de confiance. Pour aller plus loin, on pourrait ajouter OCSP, automatiser l'émission des certificats et protéger la clé racine avec un HSM.

Ce projet nous a aussi appris que la partie technique ne fait pas tout : une PKI repose beaucoup sur des règles d'organisation. Qui a le droit de demander un certificat ? Où est stockée la clé de la racine ? Que faire si une clé est perdue ? Sans réponse à ces questions, la meilleure configuration technique ne sert pas à grand-chose.

QCM – Questions 10, 13 et 18

Question 10 – Dans le mTLS, qui présente un certificat ?

Réponse C

Le client et le serveur présentent chacun un certificat afin de prouver mutuellement leur identité. Contrairement au TLS classique, où le serveur s'authentifie auprès du client, le mTLS ajoute l'authentification du client auprès du serveur.

Question 13 – Quelle commande révoque un certificat ?

Réponse C

La commande `openssl ca -revoke` permet de révoquer un certificat. Elle informe l'autorité de certification (CA) qu'un certificat ne doit plus être considéré comme valide.

Question 18 – Quelle bonne pratique protège la clé privée d'une CA ?

Réponse A

La clé privée d'une autorité de certification doit être protégée par une phrase secrète (passphrase) et des permissions d'accès strictes, par exemple `chmod 400`. Cela limite l'accès au fichier et réduit les risques d'utilisation non autorisée de la clé privée.